MARKETCAPTURE

Interview Preparation Handbook

Architecture walkthrough, questions, model answers, and live-debugging plan

For C++ systems, market data, low-latency, and HFT interviews

C++20 | NASDAQ ITCH 5.0 | MoldUDP64 | deterministic recovery | bounded-memory hot path

Your 60-second introduction

I built MarketCapture, a C++20 NASDAQ ITCH 5.0 market-data platform. It receives MoldUDP64 through socket, multishot io_uring, or DPDK paths; arbitrates redundant A/B feeds; reconstructs bounded-memory order books; records PCAP and normalized ticks asynchronously; checkpoints state; and replays deterministically. The hot path uses fixed-width events, preallocated packet storage, circular reordering, SPSC queues, and a fixed PMR arena. Correctness is protected by independent golden decoding, fuzzing, sanitizers, recovery tests, and Linux CI.

1. What is the most important invariant?

The book never receives a message after an unresolved sequence gap. Deterministic sequence order is more important than processing immediately.

2. Why not use a mutex-protected queue?

The topology is one producer and one consumer per edge. SPSC needs only acquire/release cursor publication, avoiding lock contention and unpredictable wakeups.

3. Explain acquire/release here.

The producer writes the slot, then stores head with release. The consumer loads head with acquire, so it observes the completed slot. Tail uses the symmetric pattern for reuse.

4. What happens when a queue fills?

The configured policy either rejects ingress or spins. Downstream parser fan-out yields. Counters and high-watermarks expose overload; no queue grows without bound.

5. Why is A/B arbitration necessary?

Both feeds can duplicate, delay, or lose packets independently. Arbitration emits one ordered stream, suppresses duplicates, and requests missing ranges.

6. Why copy out-of-order messages?

The NIC packet slot must be returned. A bounded fixed copy decouples lifetime without heap allocation or reference-counted packet retention.

7. Why fixed HotEvent instead of std::variant strings?

Fixed size makes queue storage predictable and avoids allocator variance. The rich typed parser remains for control-plane completeness and differential validation.

8. How does an add change the book?

Create the order-index entry and increment shares/order count at symbol-side-price. Duplicate reference, zero size, or exhausted capacity is rejected.

9. How is replace handled?

Validate old exists and new does not, remove the old remaining quantity, then insert the new reference using the old side/symbol and new price/size.

10. Why a PMR pool over monotonic allocation?

Pure monotonic memory never reuses cancelled nodes. The pool recycles same-size nodes while its fixed monotonic upstream bounds total memory.

11. How do you recover after a crash?

Load the newest checksum-valid checkpoint generation, fall back to the previous slot if needed, then replay ordered records after the saved sequence.

12. Why both PCAP and tick logs?

PCAP is wire truth for decoder/debugging; normalized ticks are compact application events for fast deterministic replay.

13. What does deterministic replay mean?

Given the same validated ordered events and checkpoint boundary, state transitions and resulting book counts are identical. Wall-clock scheduling is not claimed deterministic.

14. Why p99.9?

Rare stalls matter in low latency. A mean can look healthy while one in a thousand messages suffers a severe delay.

15. recvmmsg vs io_uring?

recvmmsg is simpler batching. Multishot io_uring amortizes submissions with provided buffers. Benchmark on the target kernel because complexity is not automatically faster.

16. When use DPDK?

When dedicated NIC queues, huge pages, CPU isolation, and operational ownership justify kernel bypass. Otherwise the kernel path may be simpler and sufficient.

17. What does DPDK compile CI prove?

API integration and build compatibility only. It does not prove physical line rate, NUMA placement, or NIC behavior.

18. How do you validate official data?

Verify NASDAQ MD5, stream-decompress BinaryFile records, run full and hot parsers, compare classification, count types, and publish the exact dataset/commit report.

19. How do fuzzing and golden tests differ?

Fuzzing explores malformed boundaries and crashes. Golden tests compare known semantics against an independent implementation.

20. Where can allocation still occur?

Construction and cold materialization allocate. Steady-state arbitration/events/book nodes use preallocated bounded storage; capacity failure is explicit.

21. What would you optimize next?

Use bare-metal flamegraphs and watermarks to select the bottleneck. Likely targets depend on active-order population, hashing, recorder pressure, or RX batching.

22. How would you debug a missing order?

Trace sequence, channel, arbitration slot, parsed type/order ID, router lookup, and prior lifecycle. Replay the PCAP with deterministic logging.

23. How do you prevent false benchmark claims?

Pin conditions, preserve raw outputs, compare same-host runs, state workload and active state size, and separate software CI from physical hardware evidence.

24. What is not complete without external access?

Licensed multicast validation, physical DPDK/FPGA results, PTP timestamp accuracy, and a published official multi-GB dataset run.

Live coding and debugging route

Build Debug, run marketcapture_threaded with a small packet count, and place breakpoints in ThreadedCaptureEngine::submit, FeedArbitrator::ingest/drain, HotItchParser::parse, ArenaBookRouter::apply, and the recorder loop. Watch expected sequence, queue size, order ID, symbol bytes, side, price, shares, and watermarks.

```
cmake -S . -B build-debug -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build build-debug
ctest --test-dir build-debug --output-on-failure
```

Whiteboard prompt

Draw: A/B NIC input -> packet pool -> arbitration -> fixed parser -> three event SPSC queues plus raw PCAP queue -> arena book / recorder / metrics / PCAP workers. Mark ownership, capacity, sequence invariant, and shutdown direction.

Questions to ask the interviewer

- What peak and burst message rates define capacity?
- How are feed gaps recovered and when is a book considered trustworthy?
- Which latency boundary is measured: NIC, parser, book, or strategy callback?
- How are CPU isolation, NUMA, IRQ affinity, and PTP managed?
- What correctness and replay evidence is required before production release?